

RISC-V SUMMIT EUROPE 2026 · DEVELOPER WORKSHOP

HW/SW Co-Design of a Multi-Guest Virtualized System

with Bao and CVA6 in Automotive

Sandro Pinto · OSYX Technologies

Joerg Seitter, Boerge Schmelz · Infineon Technologies

8 June 2026 · Palazzo dei Congressi , Bologna, Italy



Agenda

01 Foundation 10 min

Mixed-criticality, Consolidation, Automotive, Virtualization, Bao Hypervisor

02 RISC-V Virtualization primer 10 min

Hypervisor extension, two-stage MMU, AIA, SPMP & vSPMP

03 Workshop Overview. 5 min

Milestones, Prerequisites, & Outcomes

04 Milestone 0 10 min

Baseline Single-guest System w/ Baremetal App

05 Milestone 1 15 min

Single-guest System w/ Zephyr RTOS

06 Milestone 2 10 min

Dual-guest System (Zephyr + Baremetal) with Inter-VM communication

07 Wrap-up & Q&A buffer

Takeaways, Roadmap, discussion



Foundation
Mixed-criticality, Consolidation, Automotive, Virtualization, Bao Hypervisor

Mixed-Criticality Systems

Entangled Requirements

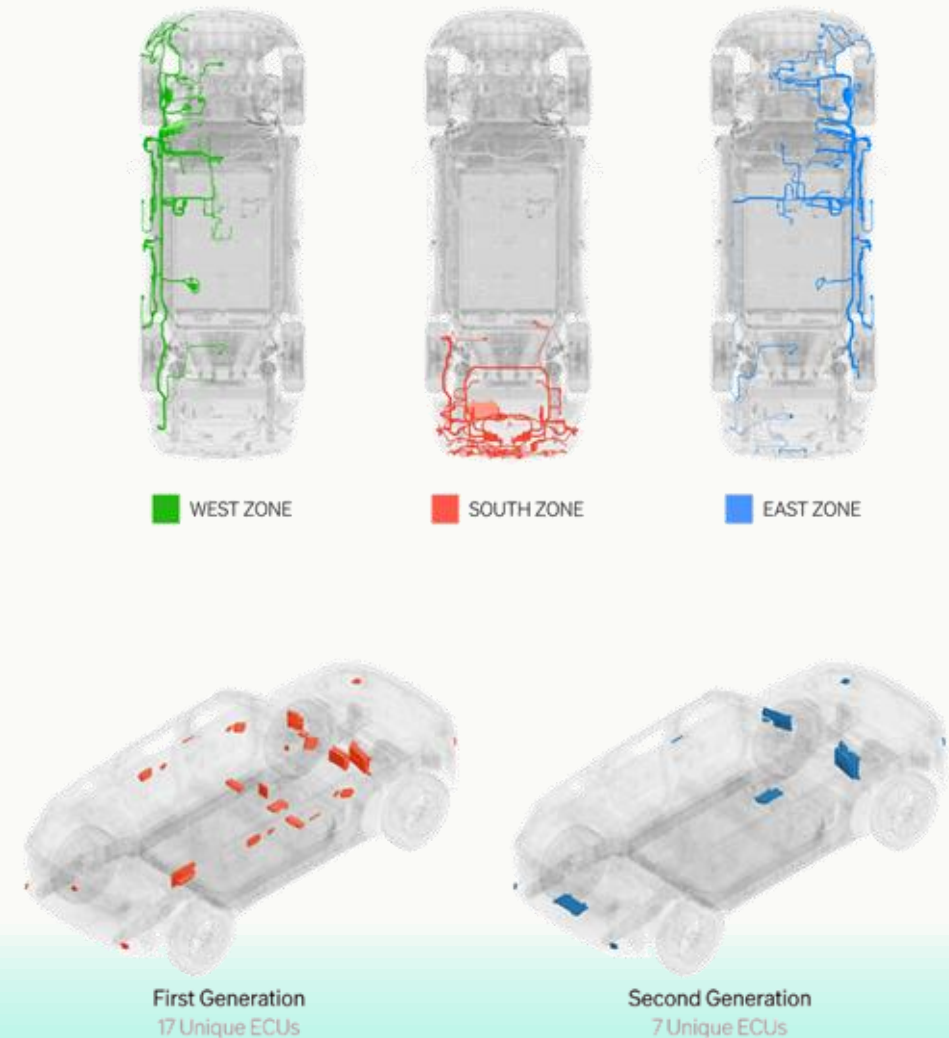


The zonal-architecture shift

The wiring harness is being replaced by a handful of zonal controllers.

- **100+ ECUs collapse into single-digit zones**
Lower cost, less wiring, less weight, faster integration and OTA.
- **One zonal MCU, many tenants**
Workloads from different teams, ASIL levels, and suppliers now share the same silicon.

Safe and secure consolidation becomes the key platform requirement.



Automotive MCUs are going mixed-criticality

- **Zonal E/E consolidation**

OEMs are collapsing 100+ single-function ECUs into a handful of zonal controllers, cutting wiring, weight, and BOM, while making the controller a shared, cross-team asset.

- **Mixed criticality on one chip**

Safety-critical control (ASIL-B/D) sits next to connectivity, diagnostics, and OTA, different teams, different ASIL levels, different suppliers, one MCU.

- **Hardware-enforced isolation, not just policy**

Safe & secure consolidation needs spatial isolation enforced by the silicon, between guests, between bus masters, and across the interrupt path. RTOS task isolation alone won't get you there (next slide).

- **And it has to stay deterministic**

Real-time bounds, certifiability (ISO 26262), and a small TCB are non-negotiable. That rules out throughput-first server-class hypervisors.

Virtualization as a key enabler for consolidation

Allows the execution of multiple operating systems on the same hardware platform. The hypervisor (VMM) provides a Virtual Machine (VM) abstraction for guest OSes.

Well-established in servers (load balancing, power management) and desktops (cross-platform, systems development).

BY DEFINITION, VIRTUALIZATION PROVIDES CONSOLIDATION

Multiple ECUs (separate chips) → **one MCU, isolated VMs**

01 Resource Management

Allocate and arbitrate CPUs, memory, devices, interrupts across VMs.

02 Abstraction

Expose a virtual-hardware interface so guests run unmodified.

03 Protection / Isolation

Confine each VM, a fault or compromise in one cannot reach the others.

Guest VM 1

Safety-critical app
(ASIL-D)

RTOS

Guest VM 2

Infotainment / cloud
(QM)

Linux

Hypervisor

Hardware (MCU + virtualization support)

A hypervisor is to an OS as an OS is to a process.

MCU virtualization is becoming a silicon-level feature

Major automotive MCU vendors are now shipping HW virtualization. Infineon AURIX TC4x, RISC-V, and other MCU ISAs all converge on the same four primitives:

- **Dedicated hypervisor privilege**

A mode above the OS kernel (HS-mode / hypervisor context) so the hypervisor isn't running as just another supervisor task.

- **Dual-stage memory protection unit**

Two-stage MPU so each guest gets its own protection view, with a hypervisor-controlled stage fencing it into a physical slice, no MMU required on MCU-class cores.

- **Interrupt virtualization w/ direct guest delivery**

Per-VM interrupt files / VMID-tagged interrupts so the hypervisor stays out of the fast path. Direct interrupt delivery to the guest, not trap-and-reinject.

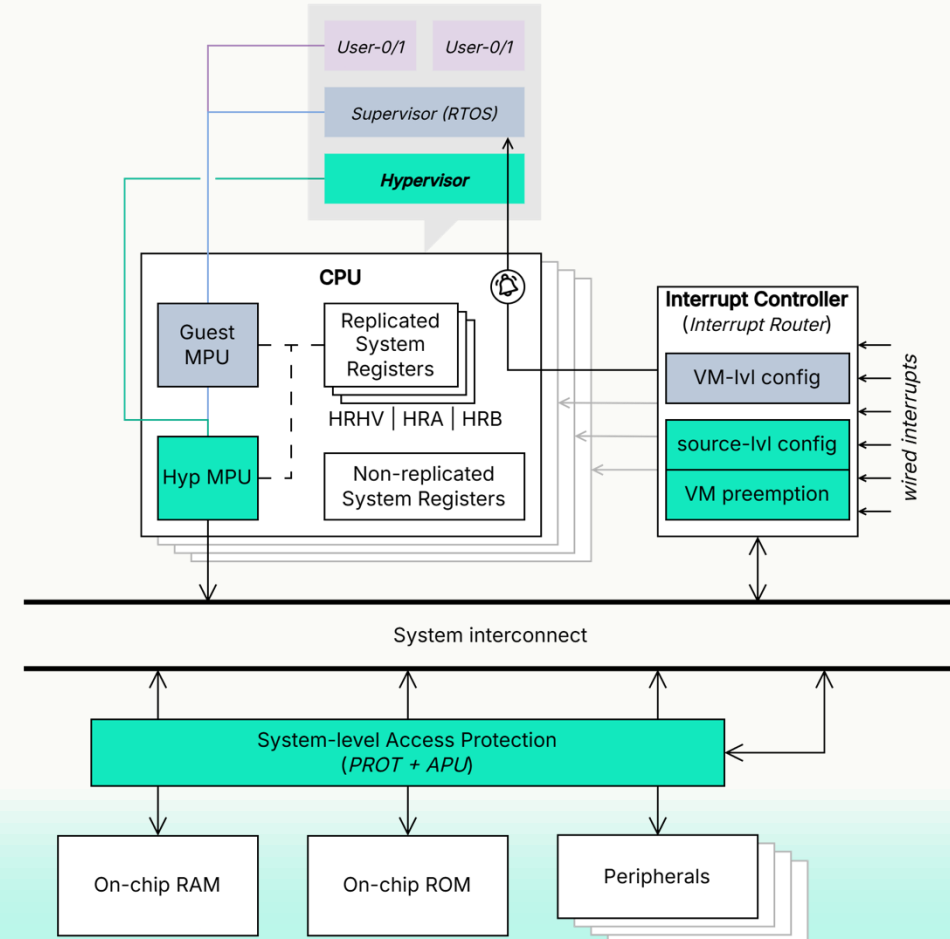
- **Peripheral access firewalls**

Bus-level enforcement (IOPMP/Worldguard) so DMA masters honour the same VM partitioning as the CPU.



Infineon TC4 Virtualization

- **Register replication**
Hypervisor (HRHV) | Real-time VMs (HRA) | Non-critical VMs (HRB)
- **Fast Context Management**
Automatic save/restore of partial context
- **Dual-stage MPU**
Hypervisor MPU + Guest MPU
Flat mapping (VA = PA)
- **Real-time Interrupt Support**
Direct interrupt routing to VMs
Priority-based handling (flat)
Inter-VM preemption support
- **System-level Access Protection**
Fine-grained memory and peripheral access controls
CPU and peripheral access mediation
Virtualization-aware

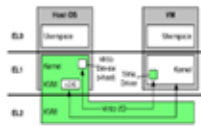


01 Traditional / Enterprise

- Large TCB (Linux-based)
- Fully-featured: networking, drivers, comm
- High-overhead I/O (emulation, paravirt)

KVM - Xen

Cloud, server, mobile



02 Embedded

- Reasonable TCB (100k–10k SLoC)
- Soft real-time
- N:1 virtual-to-physical CPU mapping

KVisor · ACRN

Generic embedded systems



03 Microkernel

- Minimal, formally verifiable design
- User-level VMMs
- Capability-based access control

seL4 - NOVA

Mobile, embodied, mixed-criticality



04 Static Partitioning

- Small TCB (5k-10k SLoC)
- No-sharing → strong isolation + real-time
- 1:1 vCPU, passthrough I/O

Bao · Jailhouse · Xen Dom0-less

Mixed-criticality systems



KVM, Xen, even most "embedded" hypervisors were built around an MMU, paged memory, and a Linux-class kernel. On an MCU-class core they fail on every axis:

- **MMU-centric design, MPU-class hardware**

Built on dynamic paging and address-space remapping. On an MPU/SPMP with a few dozen regions, the assumption that fragmentation is free doesn't survive.

- Large TCB, large attack and certification surface

Hundreds of thousands of lines of code, complex runtime services, VirtIO back-ends, networking, drivers.
Unworkable for ISO 26262 / IEC 61508 evidence.

- I/O via emulation, not assignment

Optimized for sharing one NIC across many guests, not for handing a single GPIO or CAN controller exclusively to one VM with deterministic latency.

- Tuned for throughput, not bounded latency

Schedulers, caches, and memory bandwidth are managed for average-case throughput. Mixed-criticality workloads need worst-case guarantees, the opposite optimization target.

Static partitioning hypervisors design exactly addresses this gap.

Bao Hypervisor

Static partitioning hypervisor specifically designed for mixed-criticality embedded systems.

Type-1 / bare-metal

Runs directly on hardware. No host OS in the trusted base.

Static partitioning

All resources (CPU, memory, devices) assigned at boot. No dynamic scheduling*.

Fully leverages HW virtualization support

HW privilege, dual-stage memory protection, IRQ virt, peripheral firewalls.

Minimal TCB (~10k LoC)

Small footprint amenable to safety certification.

MMU- and MPU-based systems

Same Bao across MCU (MPU-based) and MPSoC (MMU-based) silicon.

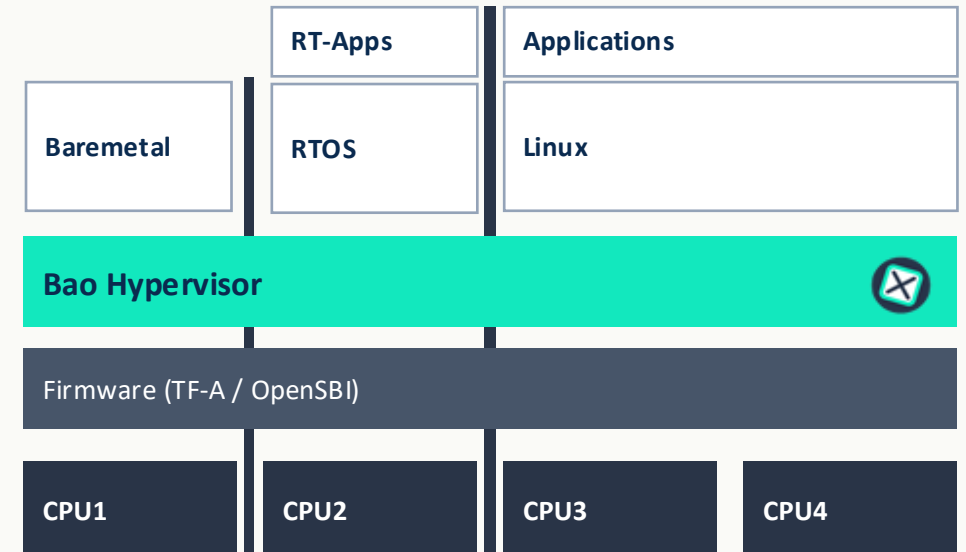
Inter-VM communication

Statically configured shared memory and asynchronous notification interrupts.

Permissive License

Apache v2.0 allows for commercial deployment without restrictions.

*Experimental CPU sharing and dynamic scheduling is work in progress and used in this workshop.



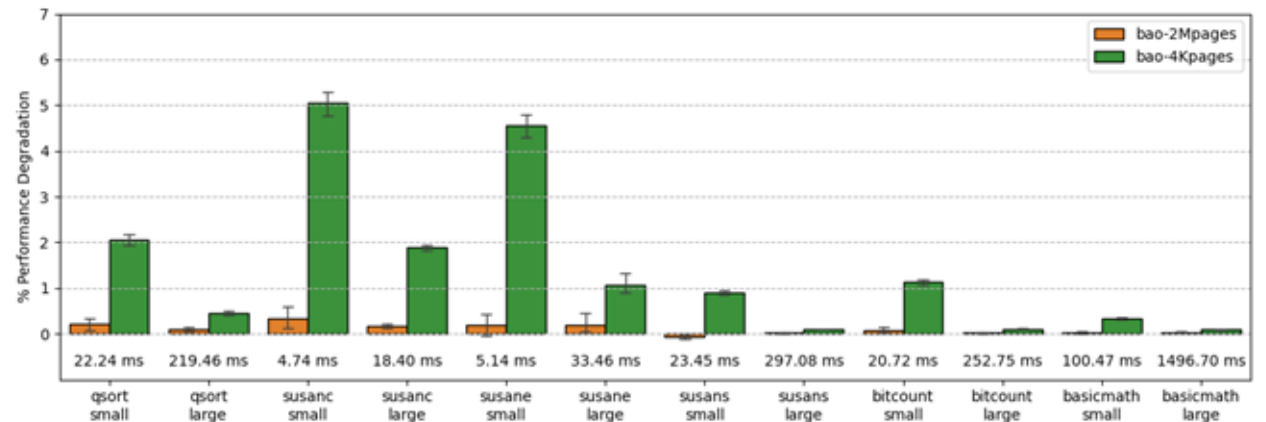
github.com/bao-project/bao-hypervisor

Minimal
Trusted Computing Base

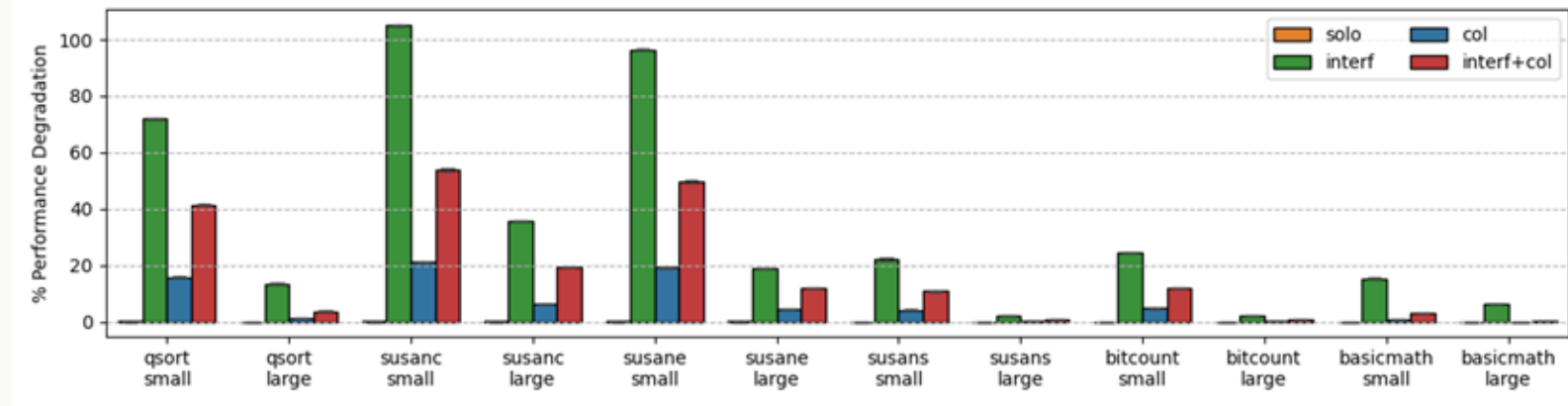
~6-9K SLoC*

*Depending on target architecture and platform

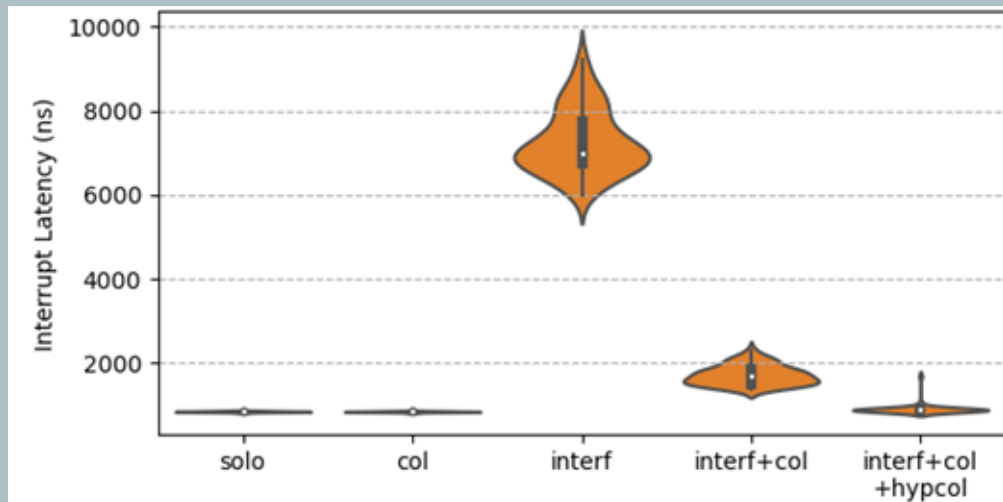
Reduced
Performance Overhead



Interference Mitigation Cache Coloring



Low IRQ Latency (sub μ s)
Interference Resistant



Bao scalability

One hypervisor, one codebase - running across architectures, platforms, and guests

Architectures

MMU-based

Armv8-A · aarch64/32
RISC-V RVH64/32

MPU-based

Armv8-R · aarch64/32
RISC-V RVH64/32 + Sspmp
Infineon TriCore TC4
Renesas RH850



Platforms

Full-blown / high-performance

Xilinx Zynq US+
NXP i.MX8
NVIDIA Jetson TX2
HiKey 960 · Toradex Verdin
Arm FVP · QEMU virt
Raspberry Pi 4
BeagleBone AI64
Rockchip RK3588
Spacemit K3 Com260

Real-time / safety-critical

NXP S32Z/E
Renesas RH850
Infineon TC4x
Semidrive E3650



Stack

Guests

Linux
Android
FreeRTOS
Zephyr
Erika III
NuttX
RT-Thread
Baremetal apps

Firmware & Tools

OpenSBI / U-Boot / EDK2
GCC + LLVM toolchains
GDB + OpenOCD
QEMU + FPGA flows



One hypervisor. One codebase. Universal embedded infrastructure.



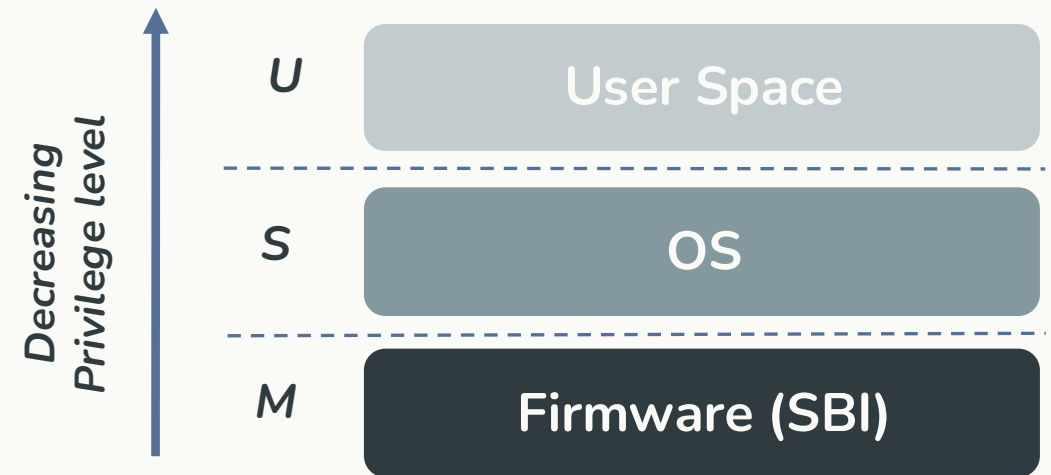
RISC-V Virtualization Prime

H- extension, 2-stage MMU, AIA, SPMP & vSPMP

RISC-V privileged architecture

Three privileged modes... and a problem when you try to virtualize

- **Machine (M-mode)**
Highest privilege dedicated to firmware.
- **Supervisor (S-mode)**
Where the OS kernel runs.
- **User (U-mode)**
Where applications run. No CSR access, no direct device access.
- **Classic virtualization is expensive here**
Although the base privileged ISA is fully virtualization, high-trap frequency required to interpose on execution of guest privileged instructions and exceptions which need to be mediated and/or emulated by the hypervisor is slow and intrusive.



The Hypervisor Extension (H)

Ratified 2021 · Promotes S-mode to HS-mode and adds two new guest modes

- **HS-mode, Hypervisor-extended Supervisor**

Where the hypervisor runs. Same instruction set as S-mode plus new CSRs (hstatus, hgap, hedeleg, hideleg, vsstatus, vsatp, ...).

- **VS-mode & VU-mode, the guest's S and U**

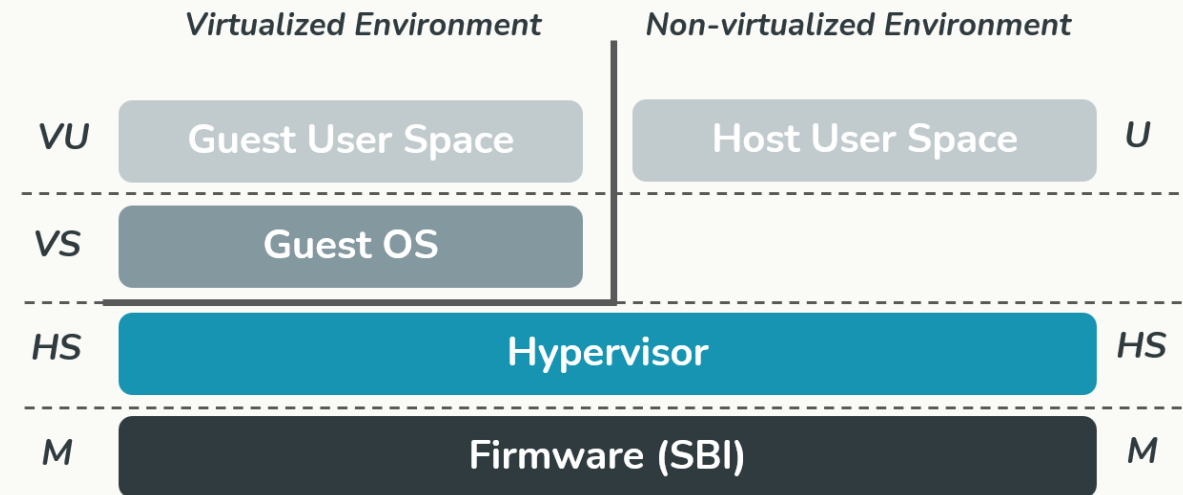
A guest kernel runs in VS-mode and uses VS-CSRs (vsatp, vsie, vstvec, ...) that look identical to S-mode CSRs from the guest's perspective.

- **Trap-and-emulate where it matters**

hedeleg / hideleg let the hypervisor pick which exceptions and interrupts go to HS-mode and which are delegated straight to the guest.

- **Why it matters**

Guest OSes (Linux, Zephyr, FreeRTOS) run unmodified at near-native speed. The hypervisor only intervenes on G-stage page faults, hypercalls, and a handful of CSR traps.



Two-stage address translation

GVA → GPA → HPA · vsatp drives stage-1, h gatp drives stage-2 (G-stage)

Two-stage Address Translation

The guest manages its own page tables exactly as on bare metal. Hypervisor only controls the second stage.

Flexible Full Virtualization

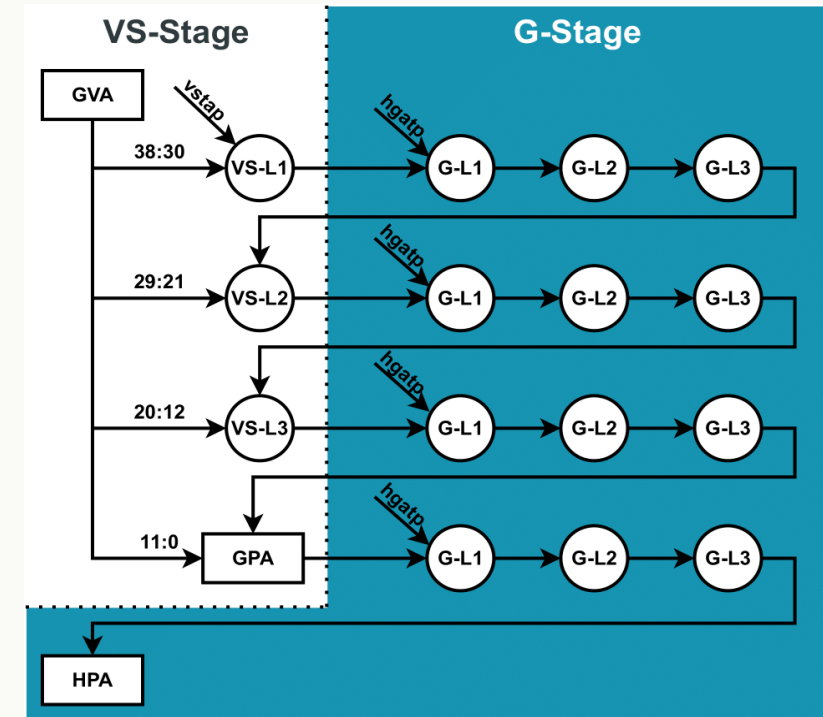
The hypervisor remaps GPA to HPA, so memory can be relocated, isolated, or overcommitted without guest awareness.

Highly Unpredictable

A TLB miss incurs in a page walk ($Sv39 \times Sv39 \times 4$) with up to 15 implicit memory accesses which might miss in the case.

Expensive Hardware and Large Memory Footprint

Requires TLBs, page-table walkers, and page-table memory.



Not suitable for hard-real time MCUs

AIA

Advanced Interrupt Architecture

- Interrupt Virtualization
- Direct Injection to Guests
- MSI Support

IOMMU

I/O Memory Management Unit

- System-level Isolation
- Address Translation
- Device Assignment



Safety
Isolation
Determinism
Bounded Latency

SPMP, Supervisor PMP

Memory isolation without an MMU, designed for MCU-class cores

- **Why not an MMU?**

Control-class cores avoid the area, power, and timing cost of a full MMU + TLB. SPMP provides spatial isolation with bounded, deterministic access checks.

- **Region-based, like PMP, but for S-mode**

S-mode owns `spmpcfgN` and `spmpaddrN`. Up to 64 regions per hart, each defines permissions, bounds, and match mode: OFF, TOR, NA4, NAPOT.

- **S-mode controlled, S- and U-mode enforced**

The OS kernel programs the regions; both the supervisor-mode kernel and user-mode tasks are checked on every load/store/fetch. No translation, just permission and bounds.

- **Deterministic**

Checks are combinational against the region table, no walk, no miss penalty. Suitable for real-time control.



RISC-V S-mode Physical Memory Protection (SPMP)

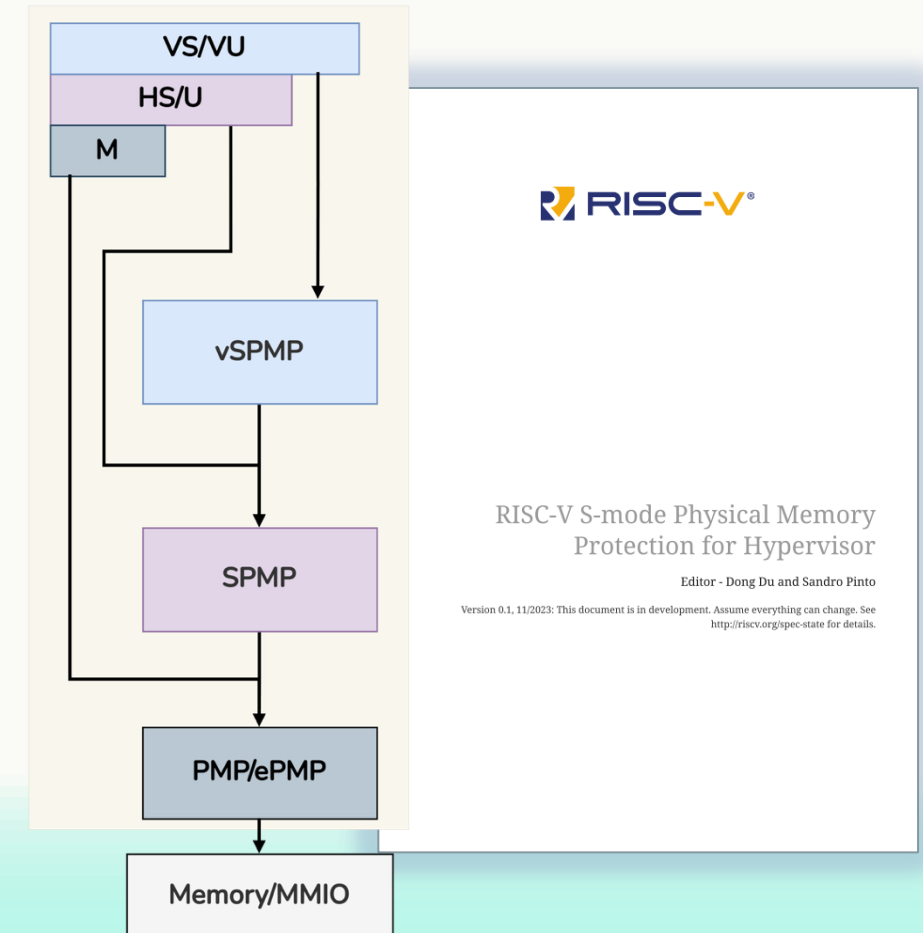
Editor - Dong Du, RISC-V SPMP Task Group

Version 0.9.1, 10/2023: This document is in development. Assume everything can change. See <http://riscv.org/spec-state> for details.

Two-stage SPMP, virtualization without an MMU

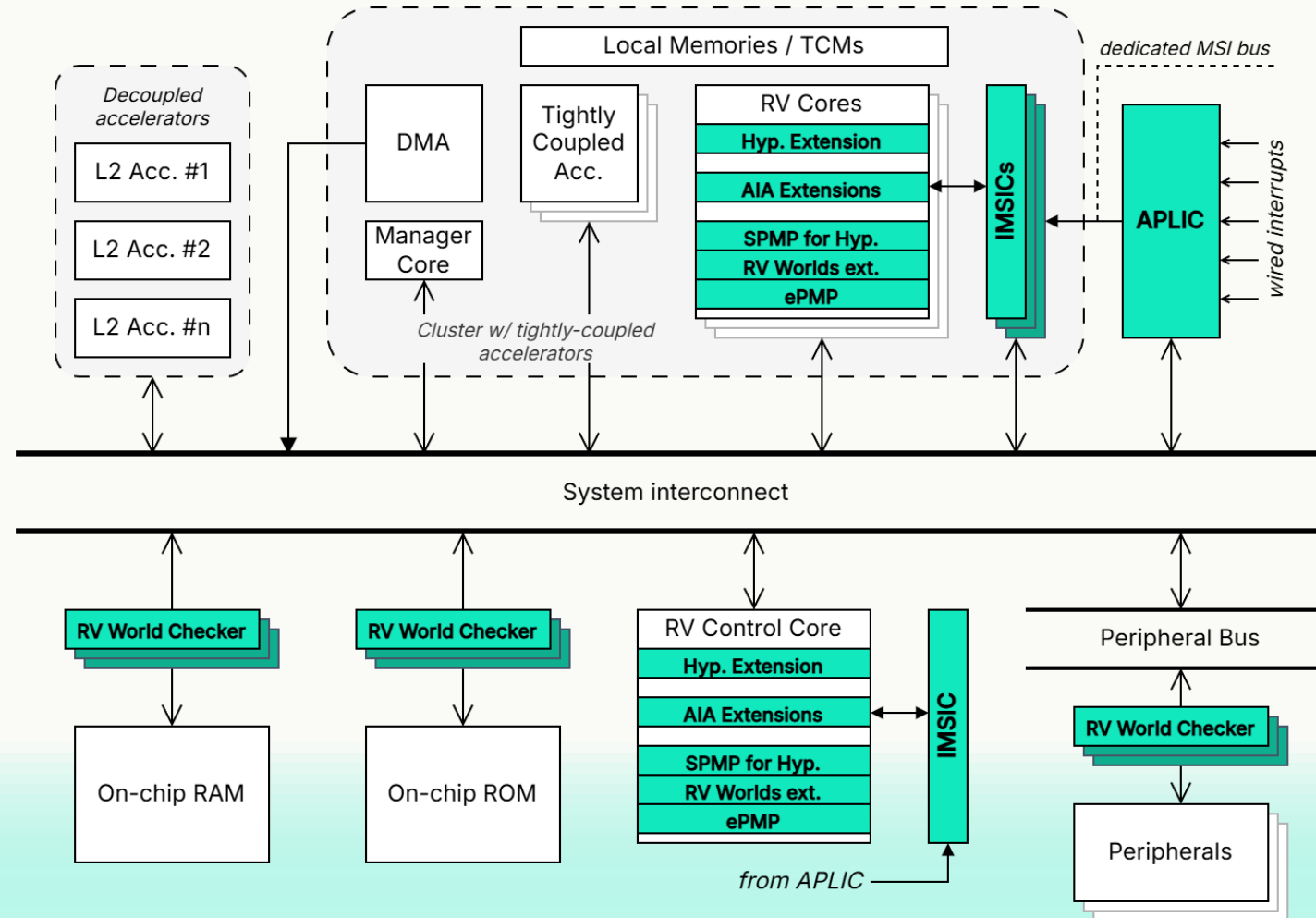
SPMP (hypervisor, G-stage) + vSPMP (guest, VS-stage), the missing piece for MCU-class virtualization

- **Hypervisor-controlled SPMP, the G-stage**
When H is implemented without an MMU, SPMP is responsible for isolation. The hypervisor programs SPMP regions to fence each guest into its own physical slice of memory and devices.
- **Guest-controlled vSPMP, the VS-stage**
Each guest is given its own vSPMP CSR bank. The guest kernel uses vspmpcfg / vspmpaddr exactly like SPMP, partitions its tasks the way a Linux/RTOS would partition its processes, with no idea it is running under a hypervisor.
- **Both checks, every access**
Loads, stores, and instruction fetches are checked against vSPMP (guest policy) AND SPMP (hypervisor policy). Either denial traps; matching grants. Still combinational, still deterministic.
- **Why this matters for the workshop?**
It's the missing ingredient that lets a static-partitioning hypervisor like Bao run on a control-class core. You'll see it live in milestone 1 when we swap to the SPMP bitstream.



RISC-V MCU Virtualization

System-level Integration (Generic)

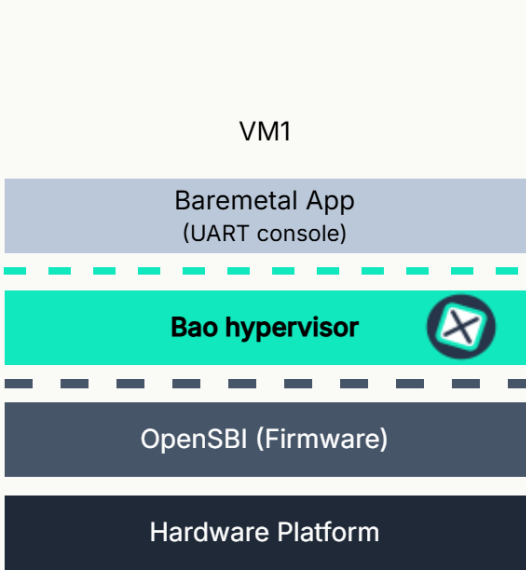


Workshop Overview

Milestones, Prerequisites & Outcomes

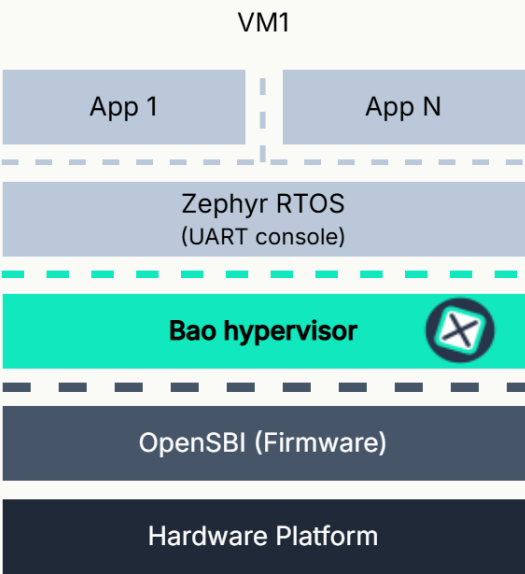
Workshop Milestones

Three incremental milestones



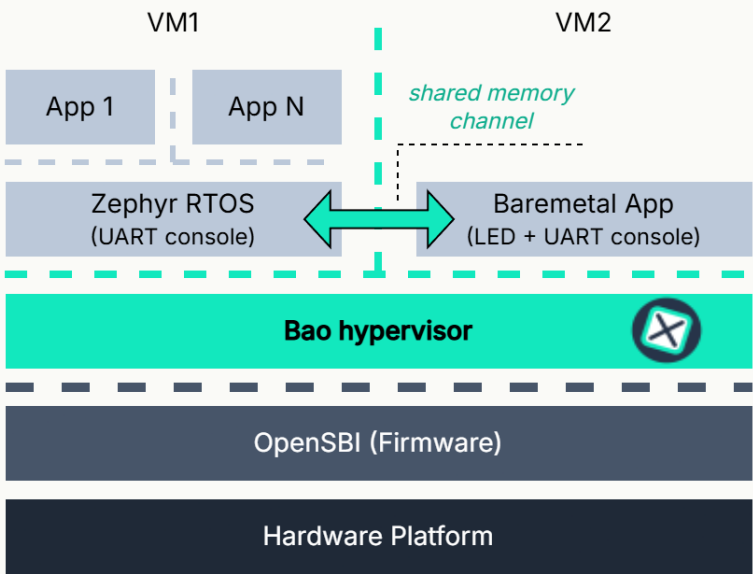
Milestone 0

Minimal single-guest baseline



Milestone 1

Swap baremetal → RTOS guest



Milestone 2

Device passthrough + inter-VM IPC + LED

Workshop Prerequisites

What you should already have

A laptop with Linux

Ubuntu 22.04/24.04 is highly recommended

Docker Engine installed and validated

Docker Engine, not Docker Desktop.

Cloned workshop repository

<https://github.com/osyxtechnologies/riscv-summit-eu-2026-workshop.git>.

Minicom terminal emulator

For serial communication.

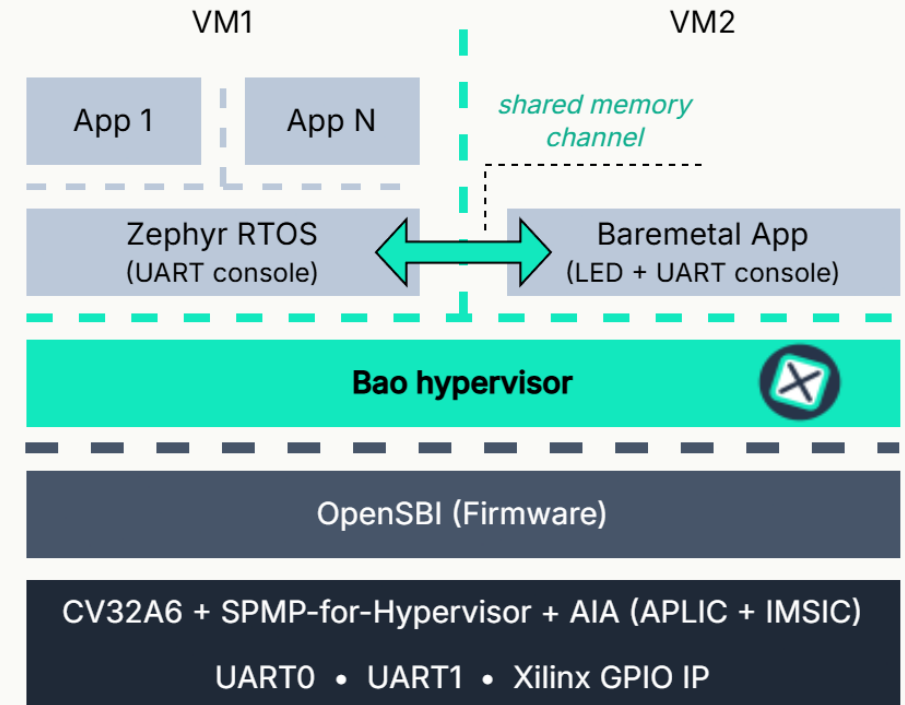
Workshop Docker image fetched/built

Contains all required SW tools.

Workshop Outcomes

What you should have at the end

- **Fully virtualized Zephyr+Baremetal system on top of Bao**
Running on real FPGA hardware or QEMU
- **Hardware-enforced isolation between VMs**
Based on RISC-V SPMP & AIA
- **Direct device and interrupt assignment**
Including LEDs and UART control
- **Inter-VM communication through shared memory**
Between two safely isolated guests
- **A fully reproducible development environment you can continue exploring after the workshop**



Software Stack & Repo Overview

What you're using, building, and where to look at

Guest applications <i>Zephyr threads</i>
Guest OS (per VM) <i>Zephyr · baremetal</i>
Bao hypervisor <i>Isolation · device passthrough · IPC · interrupt routing</i>
OpenSBI (M-mode firmware) <i>SBI calls, trap delegation</i>
CVA6 w/ SPMP/vSPMP + AIA <i>RV32IMAFCH, FPGA on Genesys 2</i>

Workshop repository

bao-hypervisor/	Bao source tree
cva6/	CVA6 source, bitstreams
exercises/	Skeleton for each milestone Bao configs
guest/	Zephyr and Baremetal guest sources
opensbi/	OpenSBI source
solutions/	The final milestone Bao configs
Dockerfile	Container image setup
Makefile	Root makefile

Hardware Platform – CVA6 w/ SPMP + AIA on Genesys 2

An open RISC-V soft-core on a Xilinx Kintex-7 board

- **CV32A6: open 32-bit application-class core**
OpenHW Group RV32 in-order pipeline, single hart on the Genesys 2 bitstream
- **Dual-stage SPMP + vSPMP**
Hardware-enforced memory partitioning without MMU, the protection Bao uses to isolate multiple VMs on a single core
- **AIA with 2 guest interrupt files**
APLIC routes wired IRQs (e.g., UART0, UART1)
IMSIC delivers IRQs directly to each guest
- **Two UARTs + GPIO IP**
UART0 on the on-board FT2232H
UART1 routed to PMOD JD
Xilinx GPIO IP wired to the on-board LEDs
- **Genesys 2: Kintex-7 dev board**
1 GB DDR3, on-board JTAG + UART interface,
8 on-board LEDs

Platform reference

Region	Base	Size
APLIC	0x0D00_0000	64 MiB
UART0	0x1000_0000	4 KiB
UART1	0x1000_2000	4 KiB
IMSIC	0x2800_0000	128 MiB
GPIO	0x4000_0000	4 KiB
DRAM	0x8000_0000	1 GiB

Interrupts

UART0: 1

UART1: 8

Workshop DRAM layout

0x8000_0000	OpenSBI + Bao
0x8420_0000	Guest #0 (Zephyr / baremetal)
0x8820_0000	Guest #1 (baremetal, m2)
0x8A00_0000	Inter-VM shmem (16 KiB, m2)

Backup Platform – QEMU RV32 w/ SPMP + AIA

Same workflow, same milestones, no FPGA needed

- **qemu-system-riscv32-smp on the virt machine**
Single hart, 4 GiB DRAM, SPMP/vSPMP and AIA enabled with 2 guest interrupt files
- **UART0 + extra UART1 via -serial pty**
UART0 bound to mon:stdio for the Zephyr guest
UART1 surfaced as a PTY for the baremetal
- **No GPIO/LED - log shim instead**
Baremetal logs "led on/off/blink" to its console where the CVA6 build would drive AXI GPIO.
- **Same OpenSBI, Bao and guests, same toolchain**
Identical exercises/ and solutions/ trees as CVA6 - only the platform constants differ.

Platform reference

Region	Base	Notes
UART1	0x0600_0000	extra (-serial pty)
APLIC	0x0D00_0000	S-mode
UART0	0x1000_0000	mon:stdio
IMSIC	0x2800_0000	S-mode MSI
DRAM	0x8000_0000	4 GiB (-m 4G)

Interrupts

UART0: 10
UART1: 12

Workshop DRAM layout (*identical to CVA6*)

0x8000_0000	OpenSBI + Bao
0x8420_0000	Guest #0 (Zephyr or baremetal)
0x8820_0000	Guest #1 (baremetal, m2)
0x8A00_0000	Inter-VM shmem (16 KiB, m2)

Important SPMP Constraints

Deterministic protection, but less flexibility than an MMU

Finite region budget

- SPMP/vSPMP entries come from a shared hardware pool with ePMP.
- On CVA6 the split between the three is fixed by the bitstream.

Milestone impact: if a VM needs too many memory or device regions, the configuration will not fit.

No address translation

- SPMP checks address bounds and permissions; it does not remap addresses.
- Guests run on the physical addresses they are actually assigned.

Milestone impact: Bao & guest images, device MMIO and shmem bases must all agree on the same physical layout.

Define your Target Platform

And stick with it for the whole workshop!

⚠️ Proceed according to your platform! ⚠️



```
export PLATFORM=cva6  
# Re-export if you close your shell
```



```
export PLATFORM=qemu  
# Re-export if you close your shell
```

Bind your Serial Nodes

/dev/ttyUSBx $\rightleftharpoons$ UARTx

 **CVA6 ONLY!** 

Identify the serial nodes assigned to the CVA6 UARTs

>_

```
ls -l /dev/serial/by-id/
```

>_

```
export CVA6_UART=/dev/ttyUSB2
```

1

```
usb-Digilent_Digilent_Adept_USB_Device_200300B0BD57-if00-port0 -> ../../ttyUSB0 # JTAG (ignore)
```

```
usb-Digilent_Digilent_Adept_USB_Device_200300B0BD57-if01-port0 -> ../../ttyUSB1 # JTAG (ignore)
```

```
usb-FTDI_FT232R_USB_UART_A904CWHH-if00-port0 -> ../../ttyUSB2 # CVA6 FTDI
```

```
usb-FTDI_FT232R_USB_UART_A50285BI-if00-port0 -> ../../ttyUSB3 # External FTDI
```

1

2

>_

```
export CVA6_UART2=/dev/ttyUSB3
```

2

Milestone 0

Baseline Single-guest System w/ Baremetal App

Milestone 0 – Setup the Hardware

15 minutes | Minimal baremetal application on top of Bao

Goal

Get a working single-guest system on the bench and *understand what is already there*

1. Flash the milestone 0 bitstream into the Genesys 2

(ignore this step if using QEMU)



```
# ONLY IF PLATFORM=cva6  
make flash CONFIG=milestone0
```

```
Jtag frequency : requested 5.00MHz -> real 5.00MHz  
Open file DONE  
Parse file DONE  
load program  
Load SRAM: [=====] 100.00%  
Done  
[flash] Done. The bitstream is live. Press RESET between runs, then 'make run'.  
[flash] Do NOT power-cycle or press PROG: both revert to the SPI-flash image (us
```

Milestone 0 – Setup the Hardware

15 minutes | Minimal baremetal application on top of Bao

2. Build all milestone 0 software artifacts

OpenSBI → Bao hypervisor → Baremetal app



```
make build CONFIG=milestone0
```

3. Reset, load, and boot

A serial console is automatically opened

RESET



```
make run CONFIG=milestone0
```

4. Check UART0 output

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 13:14:16)
Bao bare-metal guest
[heartbeat] uptime=1s
[heartbeat] uptime=2s
[uart rx] 'h'      0x68  104
[heartbeat] uptime=3s
[uart rx] 'e'      0x65  101
[uart rx] 'l'      0x6c  108
[uart rx] 'l'      0x6c  108
[heartbeat] uptime=4s
[uart rx] 'o'      0x6f  111
[heartbeat] uptime=5s
[heartbeat] uptime=6s
[heartbeat] uptime=7s
[heartbeat] uptime=8s
[heartbeat] uptime=9s
[heartbeat] uptime=10s
```

Can you see the heart beating?

To exit → 'Ctrl'+ 'A', then 'x'

Milestone 0 – Setup the Hardware

15 minutes | Minimal baremetal application on top of Bao

Goal

Get a working single-guest system on the bench... 

*...and **understand what is already there*** 

5. Open the Milestone 0 Bao configuration file

Use a code editor of your choice



```
# $EDITOR should point to your preferred code editor  
# $PLATFORM should point to your target platform  
$EDITOR exercises/$PLATFORM/milestone0/config.c
```

What do you see?

Milestone 0 – Setup the Hardware

15 minutes | Minimal baremetal application on top of Bao

.image – Where to load (or find) the guest image

`VM_IMAGE_LOADED(base addr, load addr, size)`

.entry – VM's reset vector address

In most cases, identical to image base

.cpu_num – Number of vCPUs assigned to the VM

.regions – Memory partitions assigned to the VM

Defined as *base addr + size*. Isolation is enforced by the SPMP

.devs – MMIO regions + IRQs assigned to the VM

Each entry maps a MMIO window + IRQ sources to the VM.

.arch.irqc – AIA base addresses

The APLIC and IMSIC base addresses the guest sees.

```
struct config config = {
    .vmlist_size = 1,
    .vmlist = (struct vm_config[]) {{
        .image = VM_IMAGE_LOADED(0x84200000, 0x84200000, 0x100000),
        .entry = 0x84200000,
        .platform = {
            .cpu_num = 1,
            .region_num = 1,
            .regions = (struct vm_mem_region[]) {{ /* DRAM */
                .base = 0x84200000,
                .size = 0x01000000,
            }},
            .dev_num = 1,
            .devs = (struct vm_dev_region[]) {{ /* UART0 */
                .pa = 0x10000000,
                .va = 0x10000000, // passthrough
                .size = 0x1000,
                .interrupt_num = 1,
                .interrupts = (irqid_t[]){ 1 } /* IRQ src */
            }},
            .arch = { /* AIA */
                .irqc.aplic = { .base = 0xd000000 },
                .irqc.imsic = { .base = 0x28000000 }
            },
        },
    }
};
```

exercises/\$PLATFORM/milestone0/config.c

Milestone 0 – Setup the Hardware

15 minutes | Minimal baremetal application on top of Bao

*Do you feel **capable** of writing
your **own** configuration file?*

```
struct config config = {
    .vmlist_size = 1,
    .vmlist = (struct vm_config[]) {{
        .image = VM_IMAGE_LOADED(0x84200000, 0x84200000, 0x100000),
        .entry = 0x84200000,
        .platform = {
            .cpu_num = 1,
            .region_num = 1,
            .regions = (struct vm_mem_region[]) {{ /* DRAM */
                .base = 0x84200000,
                .size = 0x01000000,
            }},
            .dev_num = 1,
            .devs = (struct vm_dev_region[]) {{ /* UART0 */
                .pa = 0x10000000,
                .va = 0x10000000, // passthrough
                .size = 0x1000,
                .interrupt_num = 1,
                .interrupts = (irqid_t[]){ 1 } /* IRQ src */
            }},
            .arch = { /* AIA */
                .irqc.aplic = { .base = 0xd000000 },
                .irqc.imsic = { .base = 0x28000000 }
            },
        },
    }
};
```

exercises/\$PLATFORM/milestone0/config.c

Milestone 1

Single-guest System w/ Zephyr RTOS

Milestone 1 – Bao Configuration File

15 minutes | Swap the baremetal application for a Zephyr RTOS guest

Goal

Write a *new Bao VM config from scratch* that replaces the baremetal guest with a *Zephyr RTOS image*

1. Write the Bao configuration file for Milestone 1

Use a code editor of your choice



```
# $EDITOR should point to your preferred code editor  
# $PLATFORM should point to your target platform  
$EDITOR exercises/$PLATFORM/milestone1/config.c
```

Milestone 1 – Bao Configuration File

15 minutes | Swap the baremetal application for a Zephyr RTOS guest

DRAM layout

0x8000_0000	OpenSBI + Bao hypervisor
0x8420_0000	Guest #0 image (Zephyr)

Platform reference

CVA6

Region	Base
APLIC	0x0D00_0000
UART0	0x1000_0000
UART1	0x1000_2000
IMSIC	0x2800_0000
GPIO	0x4000_0000
DRAM	0x8000_0000

UART0 IRQ: 1
UART1 IRQ: 8

QEMU

Region	Base
APLIC	0x0D00_0000
UART0	0x1000_0000
UART1	0x0600_0000
IMSIC	0x2800_0000
DRAM	0x8000_0000

UART0 IRQ: 10
UART1 IRQ: 12

```
struct config config = {
    .vmlist_size = 1,
    .vmlist = (struct vm_config[]) {{
        .image = VM_IMAGE_LOADED(/*TODO*/, /*TODO*/, /*TODO*/),
        .entry = /*TODO*/,
        .platform = {
            .cpu_num = 1,
            .region_num = 1,
            .regions = (struct vm_mem_region[]) {{ /* DRAM */
                .base = /*TODO*/,
                .size = /*TODO*/,
            }},
            .dev_num = 1,
            .devs = (struct vm_dev_region[]) {{ /* UART0 */
                .pa = /*TODO*/,
                .va = /*TODO*/, // passthrough
                .size = 0x1000,
                .interrupt_num = 1,
                .interrupts = (irqid_t[]){ /*TODO*/ } /* IRQ src */
            }},
            .arch = {
                /* AIA */
                .irqc.aplic = { .base = /*TODO*/ },
                .irqc.imsic = { .base = /*TODO*/ }
            },
        },
    }
};
```

exercises/\$PLATFORM/milestone1/config.c

Milestone 1 – Bao Configuration File

15 minutes | Swap the baremetal application for a Zephyr RTOS guest

2. Build all milestone 1 software artifacts

OpenSBI → Bao hypervisor → Zephyr RTOS



```
make build CONFIG=milestone1
```

3. Reset, load, and boot

A serial console is automatically opened

RESET



```
make run CONFIG=milestone1
```

4. Check UART0 output

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 13:20:28)
*** Booting Zephyr OS build 4e42fe5200d9 ***
Zephyr dual-task demo (board: baovm_cva6-smp/cv32a6)
[task_a] iter=0 (user mode)
[task_b] iter=0 (user mode)
[task_a] iter=1 (user mode)
[task_b] iter=1 (user mode)
[uart_rx] 'h'      0x68  104
[uart_rx] 'e'      0x65  101
[uart_rx] 'l'      0x6c  108
[task_a] iter=2 (user mode)
[task_b] iter=2 (user mode)
[uart_rx] 'l'      0x6c  108
[uart_rx] 'o'      0x6f  111
[task_a] iter=3 (user mode)
[task_b] iter=3 (user mode)
[task_a] iter=4 (user mode)
[task_b] iter=4 (user mode)
[task_a] iter=5 (user mode)
[task_b] iter=5 (user mode)
```

Expected Result...

Milestone 1 – Bao Configuration File

15 minutes | Swap the baremetal application for a Zephyr RTOS guest

2. Build all milestone 1 software artifacts

OpenSBI → Bao hypervisor → Zephyr RTOS



```
make build CONFIG=milestone1
```

3. Reset, load, and boot

A serial console is automatically opened

RESET



```
make run CONFIG=milestone1
```

4. Check UART0 output

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 13:14:16)
Bao bare-metal guest
[heartbeat] uptime=1s
[heartbeat] uptime=2s
[uart rx] 'h'      0x68  104
[heartbeat] uptime=3s
```

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 13:20:28)

*** ZEPHYR ERROR: insufficient vsPMP entries (kernel global entries) ***
This guest needs 2 vsPMP slots but the FPGA bitstream provides
only 0 (the hypervisor reserves the top slot for its default).
>>> Expand the hardware: flash the next milestone's bitstream. <<<
```

```
[heartbeat] uptime=6s
[heartbeat] uptime=7s
[heartbeat] uptime=8s
[heartbeat] uptime=9s
[heartbeat] uptime=10s
```

But... Zephyr Error!

Milestone 1 – Bao Configuration File

15 minutes | Swap the baremetal application for a Zephyr RTOS guest

What happened?

- *The baremetal app does not enforce internal isolation...*
- **Zephyr does!**
 - *Inter-task isolation using (virtual)SPMP*
 - **Problem:** M0 HW have no vSPMP entries

5. Increase the number of vSPMP entries

`CVA6ConfigNrPMPEntries = 40` → +8 vSPMP entries

```
// localparam CVA6ConfigNrPMPEntries = 32;
localparam CVA6ConfigNrPMPEntries = 40; // total num of PMP entries
// (M + HS + VS)

[...]

localparam CVA6ConfigMmuPresent = 0; // MMU disabled
localparam CVA6ConfigSmpPresent = 1; // SPMP enabled
localparam CVA6ConfigPMPNum = 16; // num of M-mode PMP entries
localparam CVA6ConfigPMPNumHyp = 16; // num of SPMP entries (HS)
localparam CVA6ConfigSPMPSwitchOptEn = 1;

cva6/rtl/core/include/cv32a6_imafch_smp_config_pkg.sv
```

```
Bao Hypervisor 4e42fe520 (Jun 5 2026 - 13:20:28)

*** ZEPHYR ERROR: insufficient vsPMP entries (kernel global entries) ***
This guest needs 2 vsPMP slots but the FPGA bitstream provides
only 0 (the hypervisor reserves the top slot for its default).
>>> Expand the hardware: flash the next milestone's bitstream. <<<
```



```
# $EDITOR should point to your preferred code editor
$EDITOR cva6/rtl/core/include/cv32a6_imafch_smp_config_pkg.sv
```

Milestone 1 – Bao Configuration File

15 minutes | Swap the baremetal application for a Zephyr RTOS guest

6. Now flash the milestone 1 bitstream into the Genesys2
(ignore this step if using QEMU)



```
# ONLY IF PLATFORM=cva6  
make flash CONFIG=milestone1
```

```
Jtag frequency : requested 5.00MHz -> real 5.00MHz  
Open file DONE  
Parse file DONE  
load program  
Load SRAM: [=====] 100.00%  
Done  
[flash] Done. The bitstream is live. Press RESET between runs, then 'make run'.  
[flash] Do NOT power-cycle or press PROG: both revert to the SPI-flash image (use ROM=1 to update it).
```

Milestone 1 – Bao Configuration File

15 minutes | Swap the baremetal application for a Zephyr RTOS guest

7. Repeat the workflow

Reset → Load → Boot

RESET



```
make run CONFIG=milestone1
```

8. Check UART0 output

Now it must work!

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 13:20:28)
*** Booting Zephyr OS build 4e42fe5200d9 ***
Zephyr dual-task demo (board: baovm_cva6-smp/cv32a6)
[task_a] iter=0 (user mode)
[task_b] iter=0 (user mode)
[task_a] iter=1 (user mode)
[task_b] iter=1 (user mode)
[uart rx] 'h'      0x68  104
[uart rx] 'e'      0x65  101
[uart rx] 'l'      0x6c  108
[task_a] iter=2 (user mode)
[task_b] iter=2 (user mode)
[uart rx] 'l'      0x6c  108
[uart rx] 'o'      0x6f  111
[task_a] iter=3 (user mode)
[task_b] iter=3 (user mode)
[task_a] iter=4 (user mode)
[task_b] iter=4 (user mode)
[task_a] iter=5 (user mode)
[task_b] iter=5 (user mode)
```

It's alive!

To exit → 'Ctrl'+ 'A', then 'x'

Milestone 1 – Bao Configuration File

15 minutes | Swap the baremetal application for a Zephyr RTOS guest

Falling behind?

Run the previous steps with **SOLUTIONS=1**

A. Build all milestone 1 software artifacts

```
>_ make build CONFIG=milestone1 SOLUTIONS=1
```

B. Reset, load, and boot

RESET



```
make run CONFIG=milestone1 SOLUTIONS=1
```

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 13:20:28)
*** Booting Zephyr OS build 4e42fe5200d9 ***
Zephyr dual-task demo (board: baovm_cva6-smp/cv32a6)
[task_a] iter=0 (user mode)
[task_b] iter=0 (user mode)
[task_a] iter=1 (user mode)
[task_b] iter=1 (user mode)
[uart_rx] 'h'      0x68  104
[uart_rx] 'e'      0x65  101
[uart_rx] 'l'      0x6c  108
[task_a] iter=2 (user mode)
[task_b] iter=2 (user mode)
[uart_rx] 'l'      0x6c  108
[uart_rx] 'o'      0x6f  111
[task_a] iter=3 (user mode)
[task_b] iter=3 (user mode)
[task_a] iter=4 (user mode)
[task_b] iter=4 (user mode)
[task_a] iter=5 (user mode)
[task_b] iter=5 (user mode)
```

It's alive!

To exit → 'Ctrl'+ 'A', then 'x'

Milestone 2

Dual-guest System (Zephyr + Baremetal) with Inter-VM communication

Milestone 2 – Bao Configuration File

15 minutes | Dual-guest system + Inter-VM Communication

Goal

- Add the baremetal application back as a *second isolated VM* with its own UART
- Connect the two guests with a *shared memory region + a notification IRQ*

1. Write the Bao configuration file for Milestone 2

Use a code editor of your choice



```
# $EDITOR should point to your preferred code editor  
# $PLATFORM should point to your target platform  
$EDITOR exercises/$PLATFORM/milestone1/config.c
```

Milestone 2 – Bao Configuration File

15 minutes | Dual-guest system + Inter-VM Communication

Inter-Partition Communication (IPC)

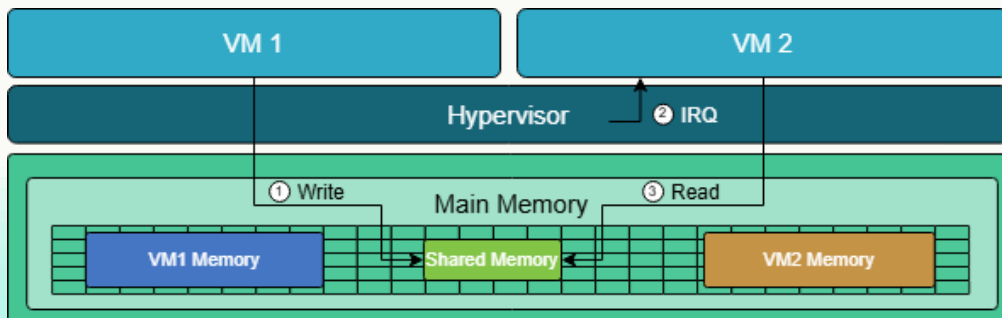
Bao provides support for IPC, allowing VMs to establish communication channels

.shmemlist – Top-level IPC configuration

One entry per shared region, declared outside any VM item

.platform.ipcs – Per-VM IPC objects specification

Each VM defines its IPC memory regions and the IRQs it uses to be notified by peers



```
struct config config = {  
    /** top-level IPC configuration **/  
    .shmemlist_size = 1,  
    .shmemlist = (struct shmem[]) {{  
        .base = /* TODO */,  
        .size = /* TODO */,  
    }},  
  
    .vmlist_size = 2, // Zephyr + Baremetal  
    .vmlist = (struct vm_config[]) {{  
        [...]  
        .platform = {  
            [...]  
  
            /** per-VM IPC configuration **/  
            .ipc_num = 1,  
            .ipcs = (struct ipc[]) {{  
                .base = /* TODO */,  
                .size = /* TODO */,  
                .shmem_id = /* TODO */,  
                .interrupt_num = 1,  
                .interrupts = (irqid_t[]) { /* TODO */},  
            }},  
  
            [...]  
        }  
    }  
};
```

exercises/\$PLATFORM/milestone2/config.c

Milestone 2 – Bao Configuration File

15 minutes | Dual-guest system + Inter-VM Communication

DRAM layout

0x8000_0000	OpenSBI + Bao hypervisor
0x8420_0000	Guest #0 image (Zephyr)
0x8820_0000	Guest #1 image (Baremetal app)
0x8A00_0000	Inter-VM shared memory region (16 KiB)

Platform reference

CVA6

Region	Base
APLIC	0x0D00_0000
UART0	0x1000_0000
UART1	0x1000_2000
IMSIC	0x2800_0000
GPIO	0x4000_0000
DRAM	0x8000_0000

UART0 IRQ: 1
UART1 IRQ: 8

QEMU

Region	Base
APLIC	0x0D00_0000
UART0	0x1000_0000
UART1	0x0600_0000
IMSIC	0x2800_0000
DRAM	0x8000_0000

UART0 IRQ: 10
UART1 IRQ: 12

```
struct config config = {
    [...]
    .vmlist_size = 2, // Zephyr + Baremetal
    .vmlist = (struct vm_config[]) {
        /*** VM1: Zephyr ***/
        .image = VM_IMAGE_LOADED(/*TODO*/, /*TODO*/, /*TODO*/),
        .entry = /*TODO*/,
        .platform = {
            .cpu_num = 1, .region_num = 1,
            .regions = (struct vm_mem_region[]) { /* DRAM */ },
            .ipc_num = 1,
            .ipcs = (struct ipc[]) { /* IPC */ },
            .dev_num = 1,
            .devs = (struct vm_dev_region[]) { /* UART0 */ },
            .arch = { /* AIA */ },
        },
    }, {
        /*** VM2: Baremetal ***/
        .image = VM_IMAGE_LOADED(/*TODO*/, /*TODO*/, /*TODO*/),
        .entry = /*TODO*/,
        .platform = {
            .cpu_num = 1, .region_num = 1,
            .regions = (struct vm_mem_region[]) { /* DRAM */ },
            .ipc_num = 1,
            .ipcs = (struct ipc[]) { /* IPC */ },
            .dev_num = 2,
            .devs = (struct vm_dev_region[]) { /* UART1, GPIO */ },
            .arch = { /* AIA */ },
        },
    },
};
```

exercises/\$PLATFORM/milestone2/config.c

Milestone 2 – Bao Configuration File

15 minutes | Dual-guest system + Inter-VM Communication

2. Build all milestone 2 software artifacts

OpenSBI → Bao hypervisor → Zephyr RTOS → Baremetal



```
make build CONFIG=milestone2
```

3. Reset, load, and boot

A serial console is automatically opened

RESET



```
make run CONFIG=milestone2
```

4. Check UART0 output

Baremetal App

```
Bao bare-metal guest
[heartbeat] uptime=1s
[heartbeat] uptime=2s
[uart rx] 'h'    0x68  104
[uart rx] 'e'    0x65  101
[heartbeat] uptime=3s
[uart rx] 'l'    0x6c  108
[uart rx] 'l'    0x6c  108
[uart rx] 'o'    0x6f  111
[heartbeat] uptime=4s
[heartbeat] uptime=5s
[heartbeat] uptime=6s
[heartbeat] uptime=7s
[heartbeat] uptime=8s
[heartbeat] uptime=9s
[heartbeat] uptime=10s
```

Zephyr

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 13:28:14)
*** Booting Zephyr OS build 4e42fe5200d9 ***
uart:~$
```

Expected Result...

Milestone 2 – Bao Configuration File

15 minutes | Dual-guest system + Inter-VM Communication

2. Build all milestone 2 software artifacts

OpenSBI → Bao hypervisor → Zephyr RTOS → Baremetal



```
make build CONFIG=milestone2
```

3. Reset, load, and boot

A serial console is automatically opened

RESET



```
make run CONFIG=milestone2
```

Baremetal App

```
Bao bare-metal guest
[heartbeat] uptime=1s
[heartbeat] uptime=2s
[uart rx] 'h'    0x68  104
[uart rx] 'e'    0x65  101
[heartbeat] uptime=3s
[uart rx] 'l'    0x6c  108
[uart rx] 'l'    0x6c  108
```

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 13:28:14)
BAO ERROR: insufficient sPMP entries: this scenario does not fit in the 16 sPM.
>>> Expand the hardware: flash the next milestone's bitstream. <<<
```

```
[heartbeat] uptime=4s
[heartbeat] uptime=5s
[heartbeat] uptime=6s
[heartbeat] uptime=7s
[heartbeat] uptime=8s
[heartbeat] uptime=9s
[heartbeat] uptime=10s
```

Zephyr

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 13:28:14)
*** Booting Zephyr OS build 4e42fe5200d9 ***
uart:~$
```

4. Check UART0 & UART1 outputs

Bao Error!

Milestone 2 – Bao Configuration File

15 minutes | Dual-guest system + Inter-VM Communication

What happened this time?

- *Bao enforces inter-VM isolation using the SPMP*
- *New VM → More entries needed*
 - *Problem: Not enough SPMP entries in M1 HW*

5. **Increase the number of SPMP entries**
+8 SPMP entries for Hypervisor use

```
// localparam CVA6ConfigNrPMPEntries = 40;
localparam CVA6ConfigNrPMPEntries = 48; // total num of PMP entries
// (M + HS + VS)

[...]

localparam CVA6ConfigMmuPresent = 0; // MMU disabled
localparam CVA6ConfigSmpPresent = 1; // SPMP enabled
localparam CVA6ConfigPMPNum = 16; // num of M-mode PMP entries
// localparam CVA6ConfigPMPNumHyp = 16;
localparam CVA6ConfigPMPNumHyp = 24; // num of SPMP entries (HS)
localparam CVA6ConfigSPMPSwitchOptEn = 1;

cva6/rtl/core/include/cv32a6_ima_fch_smp_config_pkg.sv
```

```
Bao Hypervisor 4e42fe520 (Jun 5 2026 - 13:28:14)
BAO ERROR: insufficient sPMP entries: this scenario does not fit in the 16 sPMP.
>>> Expand the hardware: flash the next milestone's bitstream. <<<
```



```
# $EDITOR should point to your preferred code editor
$EDITOR cva6/rtl/core/include/cv32a6_ima_fch_smp_config_pkg.sv
```


Milestone 2 – Bao Configuration File

15 minutes | Dual-guest system + Inter-VM Communication

6. Now flash the milestone 2 bitstream into the Genesys2
(ignore this step if using QEMU)



```
# ONLY IF PLATFORM=cva6  
make flash CONFIG=milestone2
```

```
Jtag frequency : requested 5.00MHz  -> real 5.00MHz  
Open file DONE  
Parse file DONE  
load program  
Load SRAM: [=====] 100.00%  
Done  
[flash] Done. The bitstream is live. Press RESET between runs, then 'make run'.  
[flash] Do NOT power-cycle or press PROG: both revert to the SPI-flash image (use ROM=1 to update it).
```

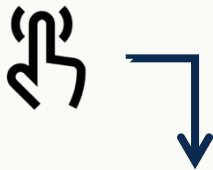
Milestone 2 – Bao Configuration File

15 minutes | Dual-guest system + Inter-VM Communication

7. Repeat the workflow

Reset → Load → Boot

RESET



```
make run CONFIG=milestone2
```

8. Check UART0 & UART1 outputs

Now it must work!

Zephyr RTOS (UART0)

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 14:45:55)
*** Booting Zephyr OS build 4e42fe5200d9 ***

uart:~$ kernel version
Zephyr version 4.4.99
uart:~$ device list
devices:
- bao-ipc@8a000000 (READY)
- aplic@d0000000 (READY)
  DT node labels: aplic
- imsic@28000000 (READY)
  DT node labels: imsic
- serial@10000000 (READY)
  DT node labels: uart0
uart:~$
```

Baremetal App (UART1)

```
Bao bare-metal guest
[heartbeat] uptime=1s
[heartbeat] uptime=2s
[uart rx] 'h'    0x68  104
[uart rx] 'e'    0x65  101
[heartbeat] uptime=3s
[uart rx] 'l'    0x6c  108
[uart rx] 'l'    0x6c  108
[uart rx] 'o'    0x6f  111
[heartbeat] uptime=4s
[heartbeat] uptime=5s
[heartbeat] uptime=6s
[heartbeat] uptime=7s
[heartbeat] uptime=8s
[heartbeat] uptime=9s
[heartbeat] uptime=10s
```

Two VMs running on one CPU!

Milestone 2 – Bao Configuration File

15 minutes | Dual-guest system + Inter-VM Communication

9. Test the IPC mechanism

Send commands to the Baremetal VM



```
# send a short message
baoipc write_notify 0 "Hello!"
# turn on the on-board LEDs
baoipc write_notify 0 "led on"
# turn off the on-board LEDs
baoipc write_notify 0 "led off"
# blink the on-board LEDs
baoipc write_notify 0 "led blink"
```

Zephyr RTOS (UART0)

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 14:45:55)
*** Booting Zephyr OS build 4e42fe5200d9 ***

uart:~$ kernel version
Zephyr version 4.4.99
uart:~$ device list
devices:
- bao-ipc@8a000000 (READY)
- aplic@d0000000 (READY)
  DT node labels: aplic
- imsic@28000000 (READY)
  DT node labels: imsic
- serial@10000000 (READY)
  DT node labels: uart0

uart:~$ baoipc write_notify 0 "Hello!"
uart:~$ baoipc write_notify 0 "led on"
uart:~$ baoipc write_notify 0 "led off"
uart:~$ baoipc write_notify 0 "led blink"
```

Baremetal App (UART1)

```
[heartbeat] uptime=110s
[heartbeat] uptime=111s
[heartbeat] uptime=112s
[heartbeat] uptime=113s
[heartbeat] uptime=114s
[uart rx] 'h' 0x68 104
[heartbeat] uptime=115s
[uart rx] 'e' 0x65 101
[uart rx] 'l' 0x6c 108
[uart rx] 'l' 0x6c 108
[uart rx] 'o' 0x6f 111
[heartbeat] uptime=116s
[shmem rx] from zephyr: "Hello!"
[heartbeat] uptime=117s
[heartbeat] uptime=118s
[heartbeat] uptime=119s
[heartbeat] uptime=120s
[heartbeat] uptime=121s
[heartbeat] uptime=122s
```

Inter-VM Communication

To exit → 'Ctrl'+ 'A', then 'x'

Milestone 2 – Bao Configuration File

15 minutes | Dual-guest system + Inter-VM Communication

Falling behind?

Run the previous steps with **SOLUTIONS=1**

A. Build all milestone 2 software artifacts



```
make build CONFIG=milestone2 SOLUTIONS=1
```

B. Reset, load, and boot

RESET



```
make run CONFIG=milestone2 SOLUTIONS=1
```

Zephyr RTOS (UART0)

```
Bao Hypervisor 4e42fe520 (Jun  5 2026 - 14:45:55)
*** Booting Zephyr OS build 4e42fe5200d9 ***

uart:~$ kernel version
Zephyr version 4.4.99
uart:~$ device list
devices:
- bao-ipc@8a000000 (READY)
- aplic@0d000000 (READY)
  DT node labels: aplic
- imsic@28000000 (READY)
  DT node labels: imsic
- serial@10000000 (READY)
  DT node labels: uart0
uart:~$
uart:~$ baoipc write_notify 0 "Hello!"
uart:~$ baoipc write_notify 0 "led on"
uart:~$ baoipc write_notify 0 "led off"
uart:~$ baoipc write_notify 0 "led blink"
```

Baremetal App (UART1)

```
[heartbeat] uptime=110s
[heartbeat] uptime=111s
[heartbeat] uptime=112s
[heartbeat] uptime=113s
[heartbeat] uptime=114s
[uart rx] 'h' 0x68 104
[heartbeat] uptime=115s
[uart rx] 'e' 0x65 101
[uart rx] 'l' 0x6c 108
[uart rx] 'l' 0x6c 108
[uart rx] 'o' 0x6f 111
[heartbeat] uptime=116s
[heartbeat] uptime=117s
[shmem rx] from zephyr: "Hello!"
[heartbeat] uptime=118s
[heartbeat] uptime=119s
[heartbeat] uptime=120s
[heartbeat] uptime=121s
[heartbeat] uptime=122s
```

To exit → 'Ctrl'+ 'A', then 'x'

Key Takeaway

Workshop Summary

What you've built and where to go next

Workshop Summary

What you've built and where to go next

What you've done today

- Built and booted a multi-guest system on top of Bao hypervisor sharing a single MCU
- Learned how SPMP-based hardware-enforced isolation enabled MMU-less virtualization in RISC-V
- Configured Bao to host single- and multi-guest workloads, including a baremetal app and an RTOS
- Enabled safe communication between two isolated guests through an IPC mechanism

Where to go next

- Add more VMs and device assignments to the Bao configuration
- Explore other MCU-based architectures supported by Bao (e.g., Infineon TC4, Renesas RH850, Armv8-R)
- Deploy an MMU-based virtualized RISC-V system and compare it with the one built today
- Explore other emerging RISC-V virtualization technologies (e.g., RISC-V Worlds, AIA, IOMMU)

THANK YOU Questions?

Sandro Pinto · sandro@osyx.tech | Joerg Seitter · joerg.seitter@infineon.com | Boerge Schmelz · boerge.schmelz@infineon.com

